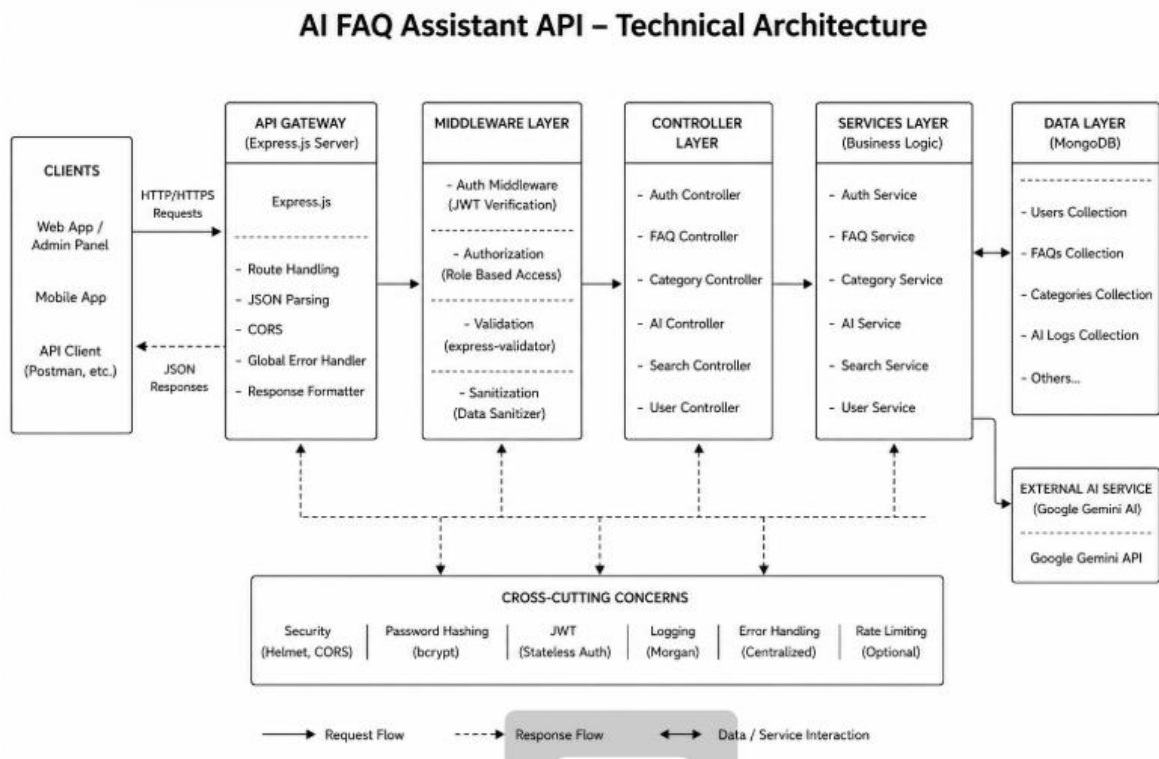


AI FAQ ASSISTANT API

3. Project Design Phase

3.1 Project Architecture



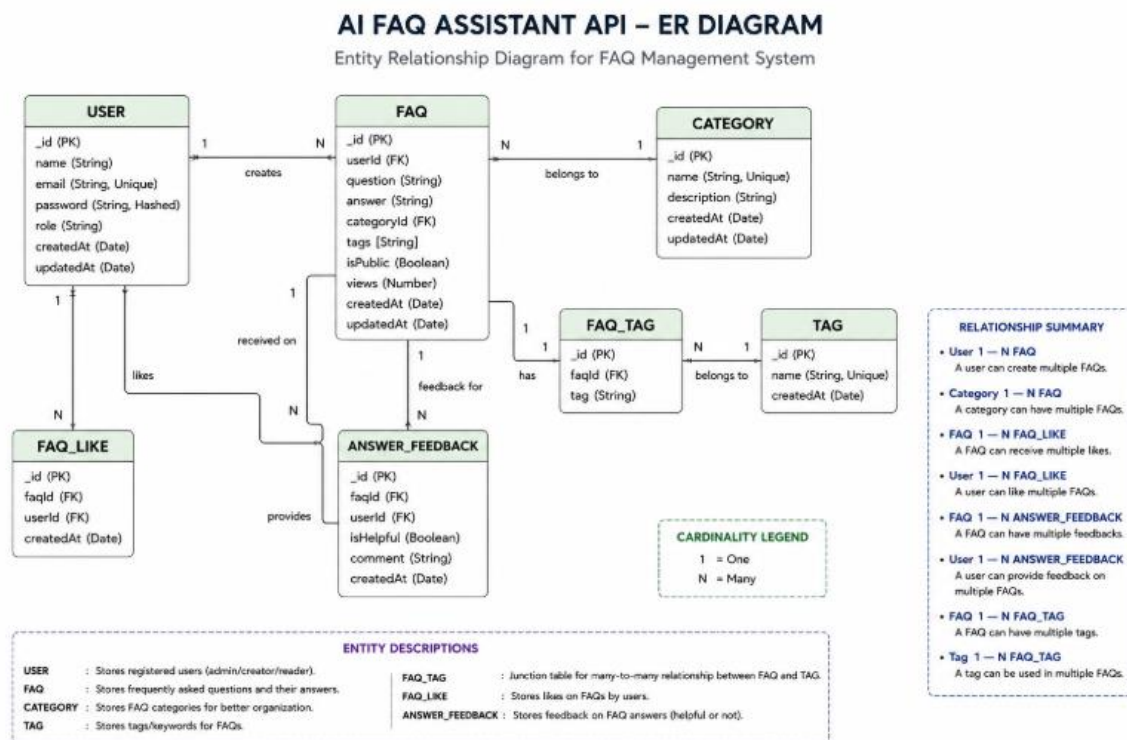
The AI FAQ Assistant application exposes structured endpoints handled by a layered server architecture model. Incoming requests undergo filter processing before interacting with data services.

3.2 project Modules

- Interface: API tools like Postman query distinct route paths to extract JSON entities.
- Express Server Gateway: Monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses incoming JSON payloads, and handles CORS headers.
- Authentication Middleware: Extracts tokens from headers, validates user context signatures, and evaluates authorization thresholds for guarded resource paths.

- Granular Services (Logic Modules): Enforces isolated domain rules for user session life cycles and operational query modifications.
- Database Interface Layer: Mongoose translates operational application logic into performant native MongoDB queries.
- AI Flow (Service Integration): Connected to external AI models via Google Gemini SDK integrations, letting the controller layer offload complex question-answering tasks and automated topic generation workflows.

3.3 Entity-Relationship (ER) Diagram Description



The backend database consists of multiple MongoDB collections connected through logical references to maintain user authentication, FAQ management, AI-generated content, and category organization.

1. User Entity

- `_id` (Primary Key - ObjectId)
- `name` (String, Required)
- `email` (String, Required, Unique)
- `password` (String, Hashed, Required)
- `role` (String, Default: "user")

- createdAt
- updatedAt

2. FAQ Entity

- _id (Primary Key - ObjectId)
- question (String, Required)
- answer (String, Required)
- category (String, Required)
- createdBy (Foreign Key referencing User Collection)
- createdAt
- updatedAt

3. AI Generation Entity (Logical)

(Generated dynamically using Google Gemini AI before persistence.)

- topic (String)
- generatedQuestion (String)
- generatedAnswer (String)
- generatedCategory (String)
- generatedAt

4. Category Entity (Logical)

- categoryName (String)
- description (String)
- totalFAQs (Number)

Key Features

- **JWT Authentication & Authorization:** Secure token-based authentication protects private API endpoints.
- **AI FAQ Generation:** Google Gemini AI automatically generates questions, answers, and topic suggestions.

- **Semantic FAQ Search:** Intelligent keyword-based search enables faster information retrieval.
- **Complete CRUD Operations:** Create, Read, Update, and Delete FAQ records.
- **Category Management:** Organizes FAQs into structured categories for easier navigation.
- **Password Encryption:** User passwords are securely hashed using bcrypt before storage.
- **Centralized Error Handling:** Standardized API error responses improve reliability and debugging.
- **Schema Validation:** Mongoose validation ensures data integrity before database persistence.
- **RESTful API Architecture:** Modular Express.js routes provide scalable backend services.
- **MongoDB Integration:** Efficient NoSQL document storage using Mongoose ODM.

Roles and Responsibilities

Admin

- Manage all registered users.
- View, update, and delete any FAQ.
- Monitor AI-generated content.
- Manage FAQ categories.
- Access complete system functionality.

Content Creator

- Create new FAQs.
- Edit personal FAQ records.
- Delete owned FAQs.
- Generate FAQs using AI.
- Organize FAQs under categories.

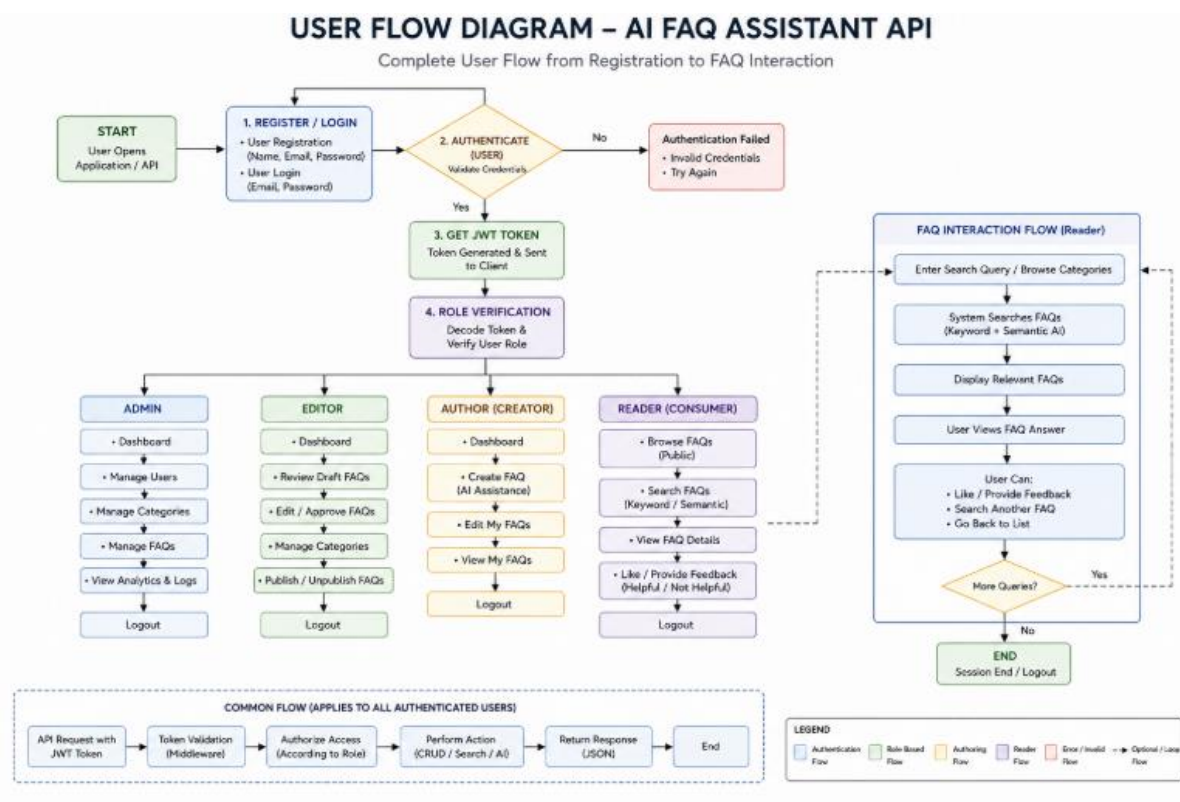
Authenticated User

- View public FAQs.
- Search FAQs using keywords.
- Generate AI-assisted answers.
- Access personal profile information.

Public User

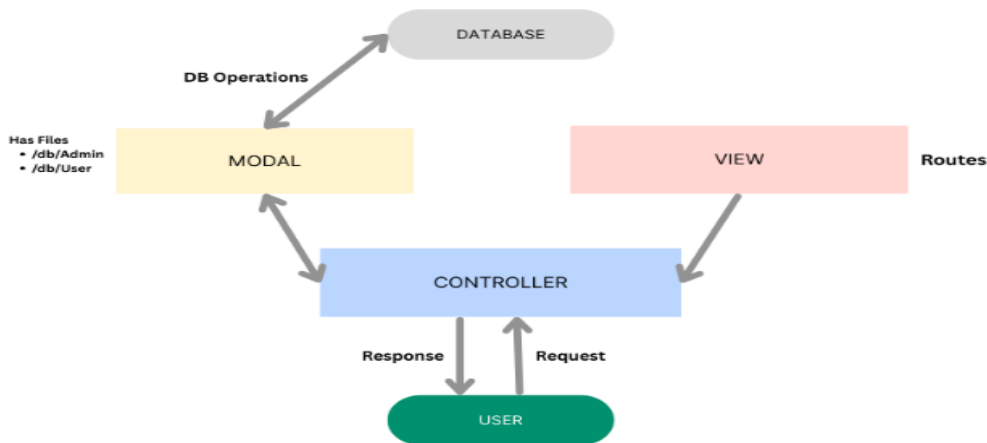
- View published FAQs.
- Search FAQs.
- Cannot create, edit, or delete FAQ records.
- No access to protected API endpoints.

User Flow



MVC Architecture Pattern : The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely

decouple components:



The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:

Model Layer (Data Blueprint)

Defines structural blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets field rules and defaults before validation persistence occurs.

Controller Layer (Execution Matrix)

The intermediary brain of the ecosystem. It captures request vectors from execution routes, checks structural request constraints, delegates processing workflows directly to models or services, and packages resultant output into standard JSON return arrays.

View Layer (Routing Network Interface)

In this pure headless backend service context, standard user-interface template views do not exist. Instead, the interface layer lives as an API Routing Layer. It links network requests on specified paths directly to their associated controller handler logic maps.